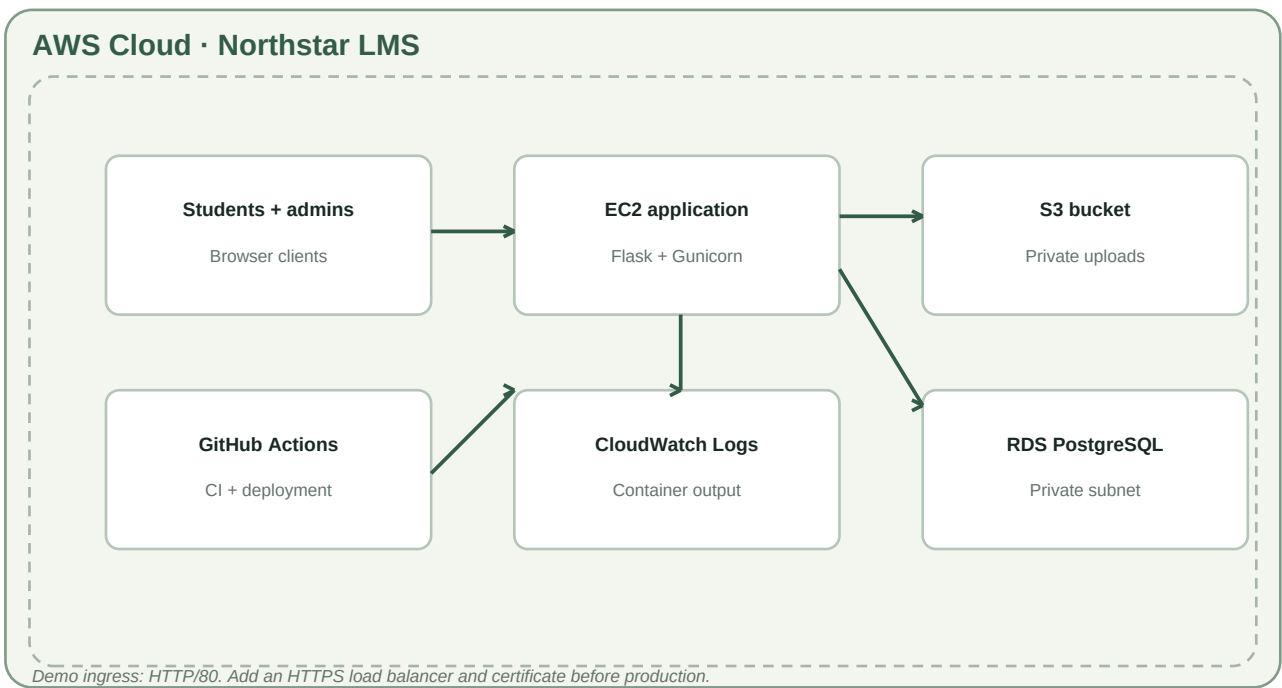


Design and Deployment of a Scalable Cloud-Based LMS

Northstar Learning Management System using AWS EC2, S3, PostgreSQL and CloudWatch



Prepared for	Project type	Platform
Cloud Engineering Submission	Student LMS	AWS · Flask · PostgreSQL

Executive Summary

This project designs and implements a cloud-hosted Learning Management System for an EdTech startup. The platform supports student registration, course discovery, private course materials, assignment submissions, and campus announcements. Administrators manage courses and publish learning content through a role-protected portal.

The implementation uses Flask and SQLAlchemy with SQLite for local development and PostgreSQL for the AWS deployment. Uploaded files use local storage in development and a private S3 bucket in the cloud. Terraform describes the VPC, EC2 application host, RDS PostgreSQL database, scoped IAM instance profile, security groups, and CloudWatch Logs group. GitHub Actions provides CI and a deployment workflow template.

Submission scope

- Complete source code with setup instructions and dependency list.
- AWS architecture diagram, infrastructure-as-code configuration, and deployment steps.
- Application screenshots and test outcomes for core local flows.
- Security and monitoring design, with cloud-only checks clearly identified as pending deployment.

Contents

Section	Topics
1	Introduction, problem statement, objectives
2	Architecture design, cloud components, data flow
3	Implementation and deployment process
4	Security controls, IAM, CloudWatch
5	Results, screenshots, test cases
6	Conclusion, learning outcomes, future enhancements

1. Introduction

Online education depends on platforms that are available across locations and can scale beyond a single campus or server room. The project brief describes an EdTech startup that needs a hosted learning platform without maintaining physical servers. The cloud engineering objective is to design the application and its supporting services, deploy the web tier, protect content, and observe service health.

Problem Statement

The startup needs a maintainable web platform where learners can access course content and submit work, while administrators can publish courses and communicate updates. A manually managed server and public file links would make content access, backup, and operational visibility difficult. A cloud design should separate compute, structured records, and uploaded objects; limit network access between them; and provide a repeatable deployment path.

Project Objectives

- Deploy a responsive LMS interface and application logic on cloud compute.
- Store relational records in PostgreSQL and learning files in S3.
- Provide student authentication and administrator authorization.
- Publish course content, announcements, assignments, and student submissions.
- Use GitHub for source control and automated build/test checks.
- Send application output to CloudWatch Logs and document useful alarms.

Success criteria

A learner can register, sign in, view a course, download a permitted material, and submit an allowed file. An administrator can create a course, add content, publish an assignment or announcement, and review submissions. The infrastructure design keeps the database private, the object bucket non-public, and application AWS permissions on an instance role. Core flows have automated tests defined against an isolated local database.

1.1 Requirements and user journeys

Actor	Primary tasks	Access boundary
Student	Register and sign in; browse courses; download course resources; submit assignment files; read announcements.	May submit own work; cannot access admin pages or admin downloads.
Administrator	Create course records; upload course materials; publish assignments and announcements; review submissions.	Role-protected management routes; admin role created through CLI.
Cloud operator	Provision infrastructure; set environment values; observe logs; patch and redeploy application.	AWS IAM identity and EC2 instance role; restricted SSH CIDR in demo.

Functional requirements

- Registration creates a student role by default; public registration never accepts an admin role.
- Login checks a password hash and uses a session cookie; logout is a CSRF-protected POST.
- A course page displays materials and assignments. Protected routes control downloads and submissions.
- Uploads use generated storage keys, an extension allowlist, and an application-level size cap.
- Announcements are visible on the landing page and signed-in dashboard.

Non-functional requirements

- Responsive layouts support desktop and narrow screens.
- Configuration comes from environment variables; secrets are excluded from version control.
- PostgreSQL and S3 are accessed through standard database and AWS SDK clients.
- Cloud resources are represented as code and can be reviewed before provisioning.

Out of scope

Payment processing, video streaming transcoding, grading workflows, external identity providers, multi-tenant organizations, and a production-grade high-availability topology are not part of this classroom prototype.

2. Architecture Design

The architecture separates the public application entry point from persistent data services. Learners and administrators connect through a browser. The Flask app runs in a Docker container on EC2. SQLAlchemy stores account, course, assignment, announcement, and submission metadata in RDS PostgreSQL. The application stores file bytes in S3 and generates short-lived signed download links after the app authorizes the request.

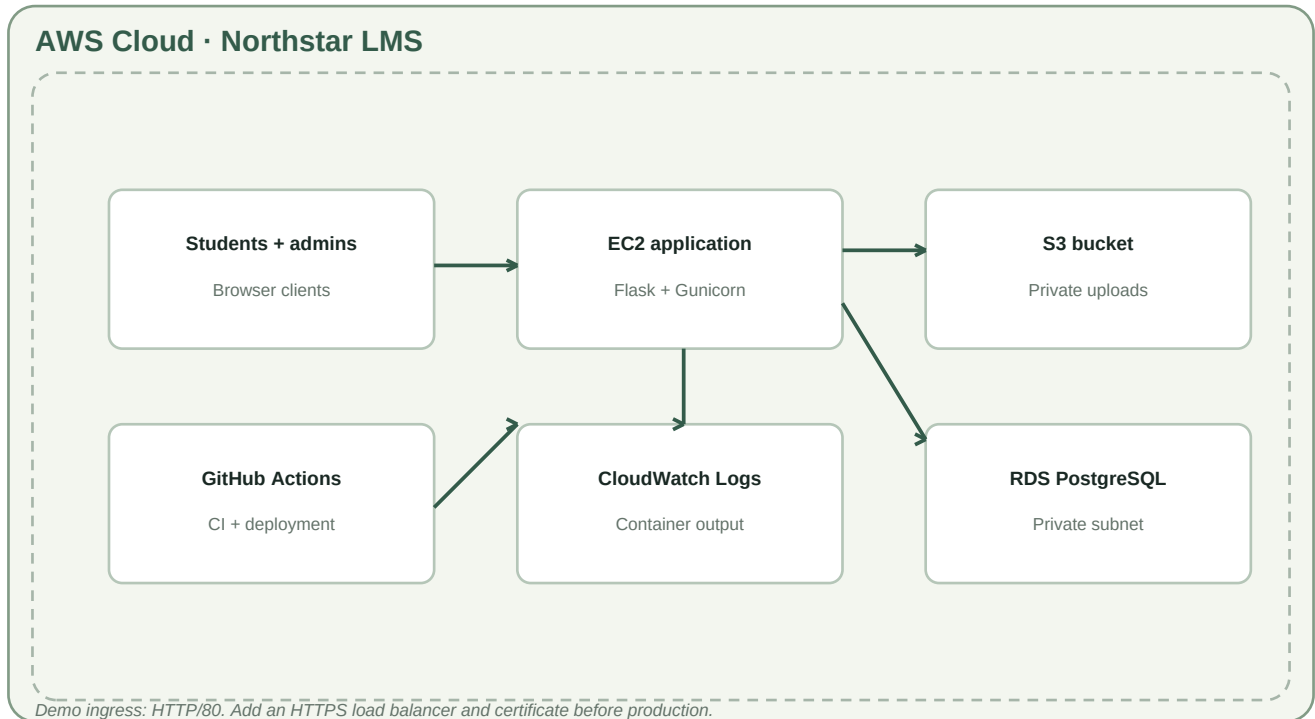


Figure 1. Reference architecture for the assignment demo. The VPC contains public EC2 ingress and isolated database subnets. CloudWatch receives Docker stdout/stderr. GitHub Actions runs CI and can trigger SSH deployment when repository secrets and host access are configured.

Network boundaries

The app security group accepts HTTP from the internet for the demo and SSH only from a caller-provided CIDR. The RDS security group accepts PostgreSQL only from the app security group. RDS is not publicly accessible and resides in private subnets. The S3 bucket blocks public ACLs and policies; only the EC2 instance role grants application access. The production evolution is to add an HTTPS load balancer and move the app into private subnets.

2.1 AWS Services and Data Flow

Service	Purpose in the design	Configuration
Amazon EC2	Hosts the Flask/Gunicorn Docker container.	Amazon Linux, encrypted root volume, IMDSv2-required instance profile.
Amazon RDS PostgreSQL	Stores user and learning-management records.	Private subnets; encrypted storage; seven-day backups; app-only SG ingress.
Amazon S3	Stores course materials and assignment files.	Private bucket, public access block, versioning, SSE-S3, role-scoped object access.
Amazon CloudWatch	Centralizes logs and operating signals.	Container logs, EC2/RDS dashboard, four metric alarms, configurable retention.
Amazon VPC	Isolates app and data networking.	Public app subnet, private DB subnets, route table and scoped SGs.
GitHub Actions	Validates changes and supports deployment.	CI on pull requests/pushes; deploy workflow uses repository SSH secrets.

Request and storage flow

- The browser sends a request to EC2. Flask checks authentication, authorization, and CSRF tokens where forms mutate data.
- Structured fields are persisted through SQLAlchemy to PostgreSQL. Database traffic stays within the VPC.
- For an upload, Flask checks filename extension and size, creates an opaque object key, and uses the instance role to write to S3.
- For a download, Flask confirms the session and material record, then returns the local file or a time-limited S3 signed URL.
- Container access and error output are shipped to CloudWatch Logs using the Docker logging driver.

Design note

A signed URL is a temporary bearer link. Keep its lifetime short and avoid logging it. For private course enrollment, extend the data model with enrollment records and verify enrollment before issuing access.

2.2 AWS Provisioning and Runtime Workflow

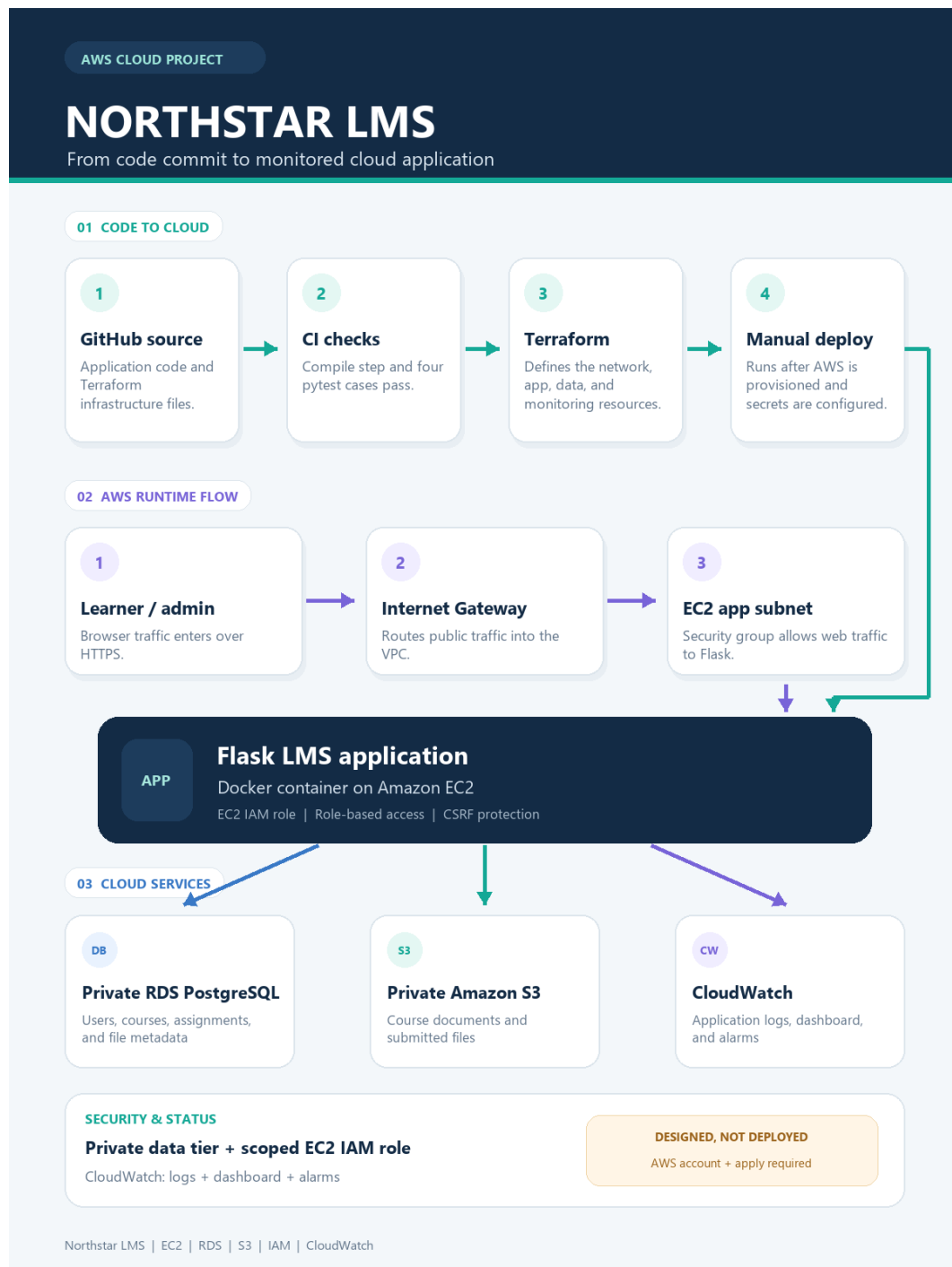


Figure 2. Delivery path from GitHub checks and Terraform to the AWS runtime, data services, and CloudWatch. The diagram represents the planned architecture; AWS resources have not been provisioned.

3. Implementation

Development Process

The application was built as a small Flask package with separate route modules, SQLAlchemy models, a storage adapter, role-aware templates, and static styles. The design keeps the local path easy to run while preserving the same service boundaries used in AWS. The test configuration uses an isolated SQLite database, temporary upload directory, and deterministic test accounts.

Application modules

Module	Responsibilities
app/__init__.py	Application factory, extensions, configuration, CLI commands, user loader.
app/models.py	Users, courses, materials, assignments, submissions, announcements.
app/auth.py	Student registration, login, logout, and session-scoped login throttling.
app/routes.py	Public catalog, learner dashboard, admin actions, course resource routes.
app/storage.py	Allowlisted upload handling and local/S3 persistence and download responses.
app/templates + static	Responsive learner and administrator screens, accessible labels, visual design.

Data model

A User has a unique email, password hash, display name, and role. Course owns Material and Assignment rows. A Submission connects one learner to one assignment and records the opaque storage key and original download name. Announcement records a publishing admin. Foreign keys and cascading relationships keep dependent content consistent when a course is removed.

Operational commands

``flask --app run.py init-db`` creates the initial tables for the prototype. ``flask --app run.py create-admin`` creates the first administrator via an operator-controlled command. These commands are intentionally not exposed as public routes. For future schema changes, adopt Alembic/Flask-Migrate instead of relying on ``create_all``.

3.1 Deployment Process

Deployment is described as a reviewed Terraform apply followed by application initialization. Terraform variables provide the AWS region, SSH source range, repository URL, and sensitive runtime values. User data installs Docker, clones the repository, writes the runtime environment file, builds the container, starts it with CloudWatch logging, and attempts initial schema creation.

Deployment sequence

- Create the GitHub repository and push the project source; set the HTTPS clone URL in Terraform variables.
- Configure an AWS identity with reviewed infrastructure permissions; generate database password and Flask secret key.
- Run Terraform init, format, validate, and plan; review estimated charges, ingress rules, subnet placement, and IAM policy.
- Apply Terraform. Wait for EC2 bootstrap and verify the application URL and CloudWatch log stream.
- Run the CLI command inside the container to create the first admin; register a learner and walk through a course flow.
- For subsequent releases, CI tests first, then the deploy workflow pulls the branch, rebuilds the image, and restarts the service.

GitHub repository settings

CI requires no write token because it only checks out code. The deployment workflow expects `EC2\_HOST`, `EC2\_USER`, and a narrowly-scoped `EC2\_SSH\_KEY` secret. The key's public half must be installed on EC2 through a deliberate access method. Terraform does not silently create or embed an SSH key pair.

Rollback and teardown

Rollback the app by redeploying a known commit. Database recovery uses RDS point-in-time recovery and a validated restore; storage versioning assists recovery of overwritten objects. Before `terraform destroy`, export any required submissions. The educational stack disables RDS deletion protection and is not a production availability design.

4. Security and IAM

Identity and access management

The EC2 instance receives an instance profile rather than static AWS access keys. The role is scoped to list the one materials bucket, read/write objects in that bucket, and write log events to the LMS log group. The application does not store long-lived AWS credentials. AWS documents the use of IAM roles to provide application credentials through instance metadata; the instance requires IMDSv2 tokens.

Application security

- Passwords are stored with Werkzeug's salted password hash functions, never as plaintext.
- Flask-Login protects session routes. Admin routes test the role. Student registration always sets `student`.
- Flask-WTF protects POST forms against cross-site request forgery. Logout is a POST action.
- Sessions use HttpOnly and SameSite=Lax cookies; secure-cookie mode is configurable for HTTPS.
- Uploads are limited by total request size and extension allowlist. Storage keys are random and independent of user filenames.
- Course files are not publicly readable. The server issues a signed URL only after the application route checks access.

Network and storage controls

RDS lives in private subnets, is not publicly accessible, and accepts inbound database traffic only from the app security group. S3 has all four public access block settings enabled and server-side encryption at rest. The EC2 root disk is encrypted. The application route exposes HTTP for assignment demonstration and must be fronted by TLS before real use.

Limitations and hardening

The demo login throttle is stored in the signed client session and is not a robust distributed limiter. Production should add a shared server-side limiter, MFA/SSO, enrollment-based resource authorization, malware scanning, secrets management, audit trails, CSP/HSTS, and restricted administrative access. Keep Terraform state encrypted because sensitive variables and rendered user data can be stored there.

4.1 Monitoring and Operations

CloudWatch logging

The Docker `awslogs` driver sends stdout and stderr from the web process to `/northstar-lms/application`. The Terraform log group has a configurable retention window. This captures access and error logs produced by Unicorn. Avoid adding passwords, signed URLs, student file contents, or session tokens to logs.

Recommended operational signals

Signal	Why it matters	Response
EC2 status checks	Detect host or system health failure.	Inspect instance/system logs; replace or recover the host.
CPU utilization	Find overload, runaway requests, or insufficient instance sizing.	Inspect traffic and logs; scale or optimize if sustained.
RDS free storage/connections	Protect relational data availability and connection capacity.	Review queries and capacity; expand storage or tune pools.
Application error count	Spot failing routes, database outages, and upload errors.	Trace request logs; roll back or patch the affected release.
S3 request failures	Detect permission, bucket, and upload configuration errors.	Verify role policy, region, and bucket configuration.

Operational routine

- Review recent error logs after each release and during incident triage.
- Review the provisioned EC2/RDS dashboard and metric alarms; attach notification actions to an SNS topic.
- Route alarm notifications to an owned on-call channel after selecting an operational contact.
- Review RDS backup status and periodically test restore procedures.
- Patch the base image and application dependencies; review IAM scope and public ingress periodically.

Deployment status

The cloud plan is ready for review and apply. No AWS resources were created for this report: an AWS account, repository URL, trusted SSH CIDR, and budget review are prerequisites. The demo HTTP endpoint must not serve real student data.

5. Results and Testing

Implemented result

The delivered source contains a responsive public course library, learner dashboard, registration/login flows, admin course and announcement tools, material upload/download, assignments and submissions, local/S3 storage adapter, PostgreSQL configuration, Terraform, Docker, and GitHub Actions workflows. The diagrams and documentation map the implemented code to the assignment cloud architecture.

Local test matrix

Test	Expected result	Status
Python source parse	No syntax errors in app and test source.	Passed
Homepage render	HTTP 200 and landing content shown.	Passed in GitHub Actions
Student registration	Session created; learner dashboard available.	Passed in GitHub Actions
Admin creates course	Course and unique code persisted.	Passed in GitHub Actions
Student views course	Course details visible after registration.	Passed in GitHub Actions
Role enforcement	Student receives HTTP 403 for admin page.	Passed in GitHub Actions
Bad credentials	Generic login rejection, no account disclosure.	Passed in GitHub Actions
Terraform validate/plan	Infrastructure parses and plan can be reviewed.	Not run; CLI unavailable
AWS S3/RDS/CloudWatch	Cloud storage, DB and logs work after provision.	Requires AWS; pending

Acceptance observations

Python syntax was parsed without importing external packages. GitHub Actions installed the dependencies, compiled the application, and completed the four pytest tests successfully on the main branch. The tests use temporary SQLite state and validate core local workflows; they do not validate a real AWS endpoint. Terraform validate/plan and a staging deployment remain separate gates.

Expected project outcome

The application code can be run locally, and the infrastructure directory documents an AWS deployment path. The cloud stack is not claimed as running until a user applies Terraform using their account. Similarly, automated GitHub actions run only after the repository is created, pushed, and supplied with any required deployment secrets.

5.1 Application Screenshots

The following static interface previews are included with the project source. They illustrate the learner-facing library, course detail, and administrator workspace; they are design previews rather than captures from a live deployed application.

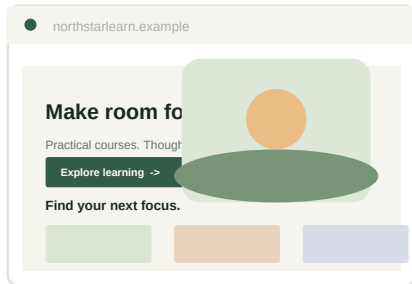


Figure 2. Course library UI preview.

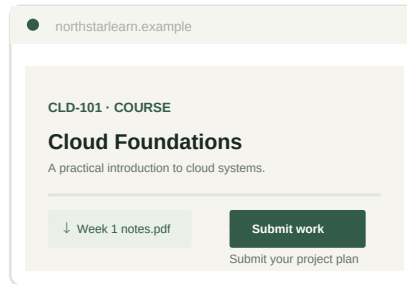


Figure 3. Materials and assignment UI preview.

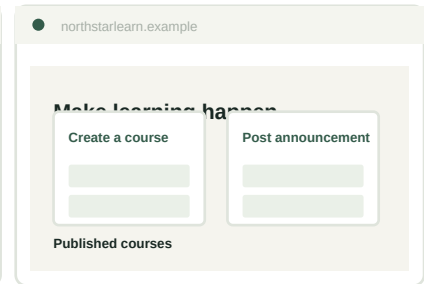


Figure 4. Admin workspace UI preview.

The full interface source lives in `app/templates/` and `app/static/styles.css`. Runtime screenshots should be captured after installing dependencies and starting the application before final classroom submission.

6. Conclusion

This project translates the assignment brief into a working LMS prototype and an AWS deployment design. The application demonstrates student and administrator workflows, relational learning records, protected content files, CI, and cloud monitoring. Terraform makes the cloud topology inspectable and repeatable, while the local SQLite setup keeps development accessible without an AWS account.

Learning outcomes

- Designed a layered cloud architecture with compute, relational storage, object storage, identity, and monitoring.
- Implemented role-aware web flows, hashed passwords, CSRF protection, file validation, and private resource storage.
- Used environment configuration, Docker packaging, infrastructure as code, and GitHub workflow automation.
- Separated verified local results from cloud activities that need account credentials and live infrastructure.

Future enhancements

- Add HTTPS Application Load Balancer, ACM, health checks, and multi-AZ app deployment.
- Adopt Alembic migrations, managed secrets, MFA/SSO, enrollment and per-course permissions.
- Add asynchronous video processing, content malware scanning, submission grading, and course progress tracking.
- Add CloudWatch dashboards and alarms with notification routing, plus structured audit logging.
- Use GitHub OIDC with a tightly-scoped deployment role or AWS Systems Manager instead of long-lived SSH credentials.
- Add Playwright end-to-end tests, accessibility review, load tests, backup restore exercises, and dependency scanning.

References

Project requirements: user-supplied four-page LMS assignment PDF and submission checklist. AWS primary documentation consulted for IAM roles on EC2, S3 public access blocking, and CloudWatch container log delivery:

- Identity and access management for Amazon EC2
- Blocking public access to Amazon S3 storage
- AWS Prescriptive Guidance: CloudWatch logging and monitoring